Task SCRUM-211: AI chatbox

Purpose of AI Use: To create an AI chatbox

Prompt: create an ai chatbot that appears for users on the client side

Response:

Added a floating CareerConnect AI chat widget that appears across client views. Users can open and close it, send messages, receive replies, and see request errors. The chat retains its conversation and sends a bounded context window.

The server now forwards validated conversations to OpenAI without exposing the API key to the client. Configure OPENAI_API_KEY in the server's .env file; OPENAI_MODEL is optional and defaults to gpt-4o-mini. Setup instructions are in README.md.

Added focused client and server tests. **Validation is incomplete:** editor diagnostics found no errors, but the test runner did not discover the tests, and shell-based lint/build/test runs were denied.

Files created
chatRoutes.js
chatAPI.js
Chatbox.jsx
Chat.integration.test.js (server/tests)
Chatbot.test.jsx (client/src/components)

Files modified
README.md
server.js
App.jsx
index.css

AI-Suggested Content: Created and updated many new files on the client-side and on the server-side. On the client-side, designed the UI for the chatbox. On the server-side, created new routes and a new integration test.

Validation: Tested manually and reviewed the files

Decision: Partially accepted

Reflection: AI helped better understand the UI design for the chatbox and how the AI chatbox works on the server-side.

Responsible Person: Johnny Quach

Task SCRUM-211: AI chatbox

Purpose of AI Use: To integrate the free web API offered by Kilo AI instead of integrating one AI model that requires an API key.

Prompt: the ai chatbot will be powered by a web api Kilo AI

Response:

The chatbot now sends conversations to Kilo AI's Gateway from the server. The Kilo API key stays server-side; the model is configurable later through KILO_MODEL rather than hardcoded. The client widget and chat API contract remain unchanged.
To enable it, set KILO_API_KEY and KILO_MODEL in the server's .env file. The README links to Kilo's API key instructions and model catalog.

Editor diagnostics found no issues. The test runner did not discover the chatbot tests, so they remain unverified.

5 files changed
README.md
chatRoutes.js
chat.integration.test.js
Chatbox.jsx
Chatbot.test.jsx

AI-Suggested Content: Modified some files on the client-side and on the server-side. On the server-side, update the routes and the integration test.

Validation: Reviewed the files and manually tested the chatbox

Decision: Partially accepted as an API key was required for the chatbox to work

Reflection: AI helped better understand how the AI chatbox works on the server-side.

Responsible Person: Johnny Quach

Task SCRUM-211: AI chatbox

Purpose of AI Use: To integrate the free web API offered by Kilo AI instead of integrating one AI model that requires an API key.

Prompt: no api keys are required. only send chat request to Kilo AI

Response:

Removed the Kilo API-key requirement. The server now sends chat requests to Kilo without an Authorization header, using the anonymous free model minimax/minimax-m2.1:free by default. You can override the model with KILO_MODEL if needed.

Updated the README and integration tests to reflect keyless requests. Editor diagnostics found no errors; I didn't run the test suite.

3 files changed
chatRoutes.js
chat.integration.test.js
README.md

AI-Suggested Content: Modified some files on the client-side and on the server-side. On the server-side, update the routes and the integration test.

Validation: Reviewed the files and manually tested the chatbox

Decision: Accepted as it implemented precisely what was asked

Reflection: AI helped better understand how the server sends chat requests to Kilo.

Responsible Person: Johnny Quach

Task SCRUM-211: AI chatbox

Purpose of AI Use: To fix the server being unable to communicate with Kilo.

Prompt: sending messages to the ai chatbot gives the message "The chat assistant could not respond. "

Response:

The chatbot was masking two different failures—Kilo rejecting the request and the server failing to reach Kilo—with the same generic message. I updated the server to report clearer causes, such as an invalid model, anonymous rate limit, denied access, or timeout, and to log Kilo's response details server-side. Added an integration test for the rate-limit case.

Restart the backend to load the change, then try again. If Kilo rejects the request, the chat should now show a more specific error. Editor diagnostics found no issues; I couldn't run the syntax checks because the shell command was denied.

3 files changed
README.md
chat.integration.test.js
chatRoutes.js

AI-Suggested Content: Modified some files on the server-side. On the server-side, update the routes and the integration test. Updated the documentation.

Validation: Reviewed the files and manually tested the chatbox

Decision: Accepted

Reflection: AI helped better understand how the server sends chat requests to Kilo.

Responsible Person: Johnny Quach

Task SCRUM-211: AI chatbox

Purpose of AI Use: To fix the response sent by Kilo.

Prompt: sending messages to the ai chatbot now gives "Kilo AI could not find the configured model. Check KILO_MODEL."

Response:

The configured model ID was no longer valid. I replaced it with Kilo's documented kilo-auto/free tier, which selects an available free model dynamically, and removed the KILO_MODEL override so a stale environment value can't override it. The request still requires no API key.

Restart the backend before testing again; it must reload the updated route in chatRoutes.js. I also updated the integration test and README. Editor diagnostics found no issues; I did not run the tests.

3 files changed
README.md
chat.integration.test.js
chatRoutes.js

AI-Suggested Content: Modified some files on the server-side. On the server-side, update the routes and the integration test. Updated the documentation.

Validation: Reviewed the files and manually tested the chatbox

Decision: Accepted

Reflection: AI helped better understand how the server sends chat requests to Kilo.

Responsible Person: Johnny Quach

Task SCRUM-211: AI chatbox

Purpose of AI Use: To fix the response sent by Kilo.

Prompt: sending messages to the ai chatbot now gives "Kilo AI returned an empty response."

Response:

Updated the Kilo response handling in chatRoutes.js: it now accepts structured text responses and allows more output tokens, which may help when a free model uses tokens for reasoning before replying. If Kilo still returns no visible text, the server now reports when it hit the output limit or returned an unsupported tool call. Added tests for structured replies and output-limit responses. **Restart the backend** before trying again. Editor diagnostics found no errors; I couldn't verify the tests with a test run. 2 files changed chat.integration.test.js chatRoutes.js

AI-Suggested Content: Modified some files on the server-side. On the server-side, update the routes and the integration test.

Validation: Reviewed the files and manually tested the chatbox

Decision: Accepted

Reflection: AI helped better understand how the server sends chat requests to Kilo.

Responsible Person: Johnny Quach

Task SCRUM-224: Build data retrieval

Purpose of AI Use: To allow the server to send data from the database to the AI chatbox.

Prompt: the ai chatbot should be able to read the data from the database sent by the server

Response:

The chatbot now receives database context from the server:

- Everyone can ask about up to **20 recent public job listings**.
- Signed-in users also get context from their own limited profile summary and up to **25 of their own recent applications**. The server verifies the session token and scopes queries to that user.
- Passwords, emails, resumes, and other users' applications aren't included. The relevant context is sent to Kilo to generate replies; the README notes this.

Updated the chat route, chat API client, and related tests. Editor diagnostics found no errors, but the test runner didn't discover the tests, so they remain unverified. Restart the backend and reload the client to try the changes.

6 files changed
chatRoutes.js
chat.integration.test.js
Chatbox.jsx
README.md
Chatbot.test.jsx
chatApi.js

AI-Suggested Content: Modified some files on the client-side and server-side. Updated the routes and the integration test.

Validation: Reviewed the files and manually tested the chatbox

Decision: Accepted

Reflection: AI helped better understand how the server retrieves and sends data to the AI chatbox.

Responsible Person: Johnny Quach

Task SCRUM-225: Implement document sharing with the AI

Purpose of AI Use: To allow users to share documents with the AI chatbox.

Prompt: implement document sharing with the AI chatbox to share any document.

Response:

Implemented document sharing in the chatbot. Users can attach **PDF, DOC, or DOCX files up to 5MB**; the server extracts up to 40,000 characters of text in memory and includes it with chat requests. The attachment stays selected for follow-up questions until removed. Uploaded files are not stored by the chat endpoint, but their extracted text is sent to Kilo AI.

Updated the chat UI, API client, server route, and README. Added tests for attaching documents and passing their text into the Kilo request.

Dependency setup is incomplete: document parsers are listed in server/package.json, but installation was denied and server/package-lock.json could not be updated. Run npm install in the server directory before starting the backend. Editor diagnostics found no errors; tests were not run.

Many files changed
README.md
Chatbot.test.jsx
chatRoutes.js
chat.integration.test.js
chatApi.js
Chatbot.jsx
package.json (server)
documentText.js

AI-Suggested Content: Modified many files on the client-side. Updated the README.md to inform about sharing documents. Installed required packages on the server-side to allow sending a document to Kilo. Created new files on the server-side.

Validation: Reviewed the files that were modified and shared sample documents to test the functionality

Decision: Accepted

Reflection: AI helped better understand how sharing documents works on the client-side and server-side.

Responsible Person: Johnny Quach

Task SCRUM-225: Implement document sharing with the AI

Purpose of AI Use: To fix an issue where Kilo could not respond within a given period

Prompt: sending the document gives the error "Kilo AI took too long to respond. Please try again."

Response:

Reduced the document request size to help prevent Kilo timeouts: the server now sends at most 12,000 extracted text characters, fewer database records, and only the six most recent chat messages when a document is attached. Document requests get a 90-second timeout; chats without documents retain their previous context and response limit. The chat UI now tells users how much document text is sent.

Validation passed: server lint, all 8 chat integration tests, the document-extraction unit test, and whitespace checks. **Restart the backend** to apply the changes. If requests still time out, Kilo's anonymous free-model service may be slow or unavailable.

5 files changed
documentText.test.js
chat.integration.test.js
chatRoutes.js
README.md
Chatbox.jsx

AI-Suggested Content: Updated the README.md to give more information about sharing documents. Created a unit test for documentText.js. Modified existing files on the client-side and server-side.

Validation: Reviewed modified files, reviewed the new unit test and shared sample documents to test the functionality

Decision: Accepted

Reflection: AI helped better understand how sharing documents works on the client-side and server-side. AI also helped realize that performance should be considered for implementing the document sharing functionality.

Responsible Person: Johnny Quach

Task SCRUM-225: Implement document sharing with the AI

Purpose of AI Use: To fix a unit test

Prompt: running "npm test:unit" causes an error

Response:

The client unit-test failure was a stale expectation in Chatbot.test.jsx: sendChatMessage now receives a third argument (null when no document is attached). I updated the test to match.

Validation passes: **39 client tests across 9 suites** and client lint. Run the script as npm run test:unit from the client directory.

1 file changed
Chatbot.test.jsx

AI-Suggested Content: Made a minor modification in the unit test Chatbot.test.jsx to fix the issue.

Validation: Reviewed the modification and ran unit tests

Decision: Accepted

Reflection: AI helped better understand how sharing documents works on the client-side.

Responsible Person: Johnny Quach

Task SCRUM-224: Build data retrieval

Purpose of AI Use: Change how the AI chatbox responds to the user when receiving the data from the server

Prompt: the ai chatbox should not comment on the data sent by the server

Response:

Updated the server's assistant instructions to use database and document context privately, without mentioning the hidden context, server, database, or that information was supplied. If the context doesn't answer a question, it should say so briefly without describing internal data sources. Added an integration-test assertion for the instruction. Server lint and all 8 chat integration tests pass. Restart the backend to apply the change. 2 files changed chatRoutes.js chat.integration.test.js

AI-Suggested Content: Made a few minor modifications in the integration test and in chatRoutes.js to fix the issue.

Validation: Reviewed the modifications and verified the output generated by the AI chatbox

Decision: Accepted

Reflection: AI helped better recognize the importance of how to give an appropriate prompt.

Responsible Person: Johnny Quach

Task SCRUM-111: AI Chatbox

Purpose of AI Use: Allow users to enter full-screen mode with the AI chatbox

Prompt: implement fullscreen for the ai chatbox

Response:

Added a fullscreen toggle to the chatbot header. It expands the chat to fill the viewport, hides the floating launcher while expanded, and lets users return to the regular panel or close the chat. The message area remains scrollable.

Editor diagnostics found no issues. I couldn't run the chatbot test because the test-run request was denied.

3 files changed
Chatbot.test.jsx
Chatbot.jsx
index.css

AI-Suggested Content: Changed the style of the AI chatbox, the unit test and implemented the full-screen mode.

Validation: Reviewed the modifications and tested the full-screen mode

Decision: Accepted

Reflection: AI helped better understand how to implement full-screen mode.

Responsible Person: Johnny Quach

Task SCRUM-225: Implement document sharing

Purpose of AI Use: To validate the format of the shared document before sending to the AI

Prompt: the AI chatbox should not allow any other formats other than the ones specificed for sharing documents.

Response:

The chatbot now accepts only **PDF, DOC, and DOCX** documents:

- The client rejects unsupported extensions, mismatched MIME types, and files over 5MB before attaching them.
- The server independently checks extension/MIME pairs and file signatures, so renaming another file type won't bypass the restriction.

Validation passed: 41 client unit tests, 9 chat integration tests, client and server lint, and git diff --check.

4 files changed
Chatbot.jsx
chat.integration.test.js
Chatbot.test.js
chatRoutes.js

AI-Suggested Content: Modified many files to implement document format validation

Validation: Reviewed the modifications and manually tested the validation by sending documents in different formats.

Decision: Accepted

Reflection: AI helped better understand how validation works for document formats.

Responsible Person: Johnny Quach